



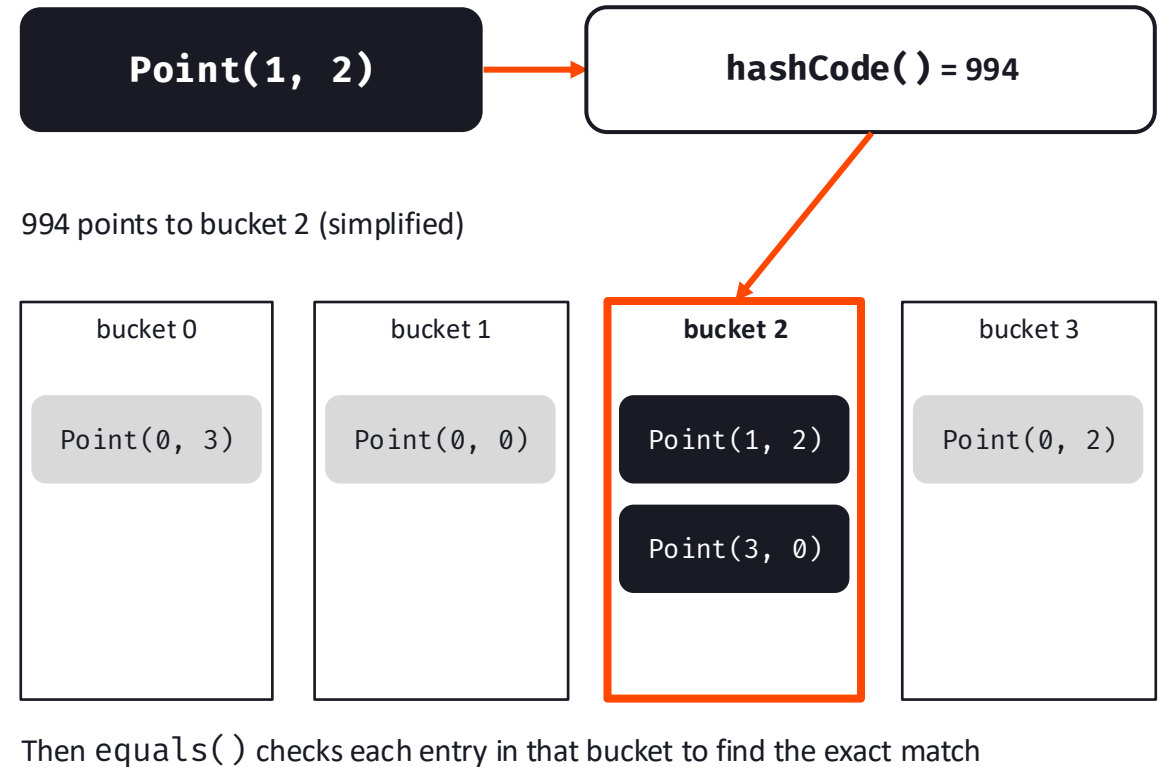
Java hashCode()

When to override it, and when to leave it alone



What is a hash code?

- A hash code is an `int` that summarises an object
- `HashMap` and `HashSet` use it to choose a bucket
- Step 1: `hashCode()` jumps straight to the right bucket (roughly $O(1)$)
- Step 2: `equals()` checks the entries in that bucket for the match
- `hashCode()` finds the neighbourhood; `equals()` finds the house
- Lists never call `hashCode()`: they only use `equals()`



The hashCode() contract

1

Equal objects, equal hash codes

If `a.equals(b)` is true, then
`a.hashCode() == b.hashCode()`
must be true.

This is the rule people break.

2

Consistent

Calling `hashCode()` again on an
unchanged object must return the same
value.

3

Collisions are allowed

Unequal objects MAY share a hash code.
Allowed, but lots of collisions hurt
performance.

Same hash code does not prove equality: `"Aa".hashCode()` and `"BB".hashCode()` are both 2112, but the strings are not equal.



The default behaviour

- Every class inherits `equals()` and `hashCode()` from `Object`
- `equals()` means identity: true only if `a == b`, the very same object
- `hashCode()` is an identity hash, usually different for each object
- You can see it with `System.identityHashCode(obj)`
- Valid and consistent: it just models "same object", not "same values"

```
Point a = new Point(1, 2);
Point b = new Point(1, 2);

a.equals(b);           // false
a.hashCode();          // e.g. 2045766957
b.hashCode();          // e.g. 690521419

Set<Point> set = new HashSet<>();
set.add(a);
set.contains(b);       // false
```

From `Step01DefaultIdentity.java`



When to override

Value objects

- Two instances with the same data are the same thing
- Override `equals()` and `hashCode()` together

The golden rule

- If you override `equals()`, you must override `hashCode()`
- Otherwise a `HashSet` stores "equal" duplicates and `map.get()` returns `null`

Typical examples

- `Point`, `Money`, `DateRange`, `Isin`, `CurrencyPair`, `Address`
- Composite map keys, such as `(accountId, tradeDate)`
- Anything in a `HashSet` to remove duplicates by content
- DTOs compared in unit tests with `assertEquals`



When to keep the default

Identity matters

- Objects with a lifecycle: tasks, jobs, sessions
- Resources: connections, threads, sockets, file handles
- Services, controllers and repositories
- UI components and event listeners
- Mutable objects that change while in a collection

Example: DownloadTask

Two downloads of the same URL are two separate jobs. Cancelling one must not cancel the other, so identity equality is exactly right.

Rule of thumb

If two had identical fields, would they be the same thing?

Yes: override. No: keep the default.



Two classic ways to write it

Option A: Objects.hash (simplest)

```
@Override
public int hashCode() {
    return Objects.hash(x, y);
}
```

- Readable and null-safe
- Creates a small array on each call: only matters in very hot code

Option B: the classic 31 * pattern

```
@Override
public int hashCode() {
    int result = Long.hashCode(amountInPence);
    result = 31 * result + currency.hashCode();
    return result;
}
```

- What IDEs generate, and how `String` does it
- 31 is an odd prime: spreads values well and is cheap to compute



Records, tools and checklist

Option C: records (best for new code)

```
record Point(int x, int y) {}
```

The compiler generates `equals()`, `hashCode()` and `toString()`.
Records are immutable too.

Option D: let the tools do it

- IntelliJ: Code > Generate > `equals()` and `hashCode()`
- Eclipse: Source > Generate `hashCode()` and `equals()`
- Lombok: `@EqualsAndHashCode` or `@Value`

Checklist

- Use exactly the same fields in `equals()` and `hashCode()`
- Only use fields that do not change while the object is in a collection
- For arrays use `Arrays.hashCode(array)`, not `array.hashCode()`
- Never return a constant: legal, but slow (see pitfalls)



Common pitfalls

Mutable keys

Change a field used in `hashCode()` after adding the object to a set and it is stranded in its old bucket. Fix: make value objects immutable.

Arrays inside records

A record compares array fields by identity, not contents. Prefer `List.copyOf(...)` for collection components.

Constant hash codes

`return 42;` is legal but puts every object in one bucket: milliseconds become seconds.

`instanceof` vs `getClass()`

`instanceof` is fine for final classes and records. `getClass()` is stricter and safer in class hierarchies.

JPA / Hibernate entities

The ID is often null until saved. Do not generate from all fields; follow your team convention.

Remember

Override both, together.
Keep keys immutable.
Prefer records for new value types.



Thank you

neueda 